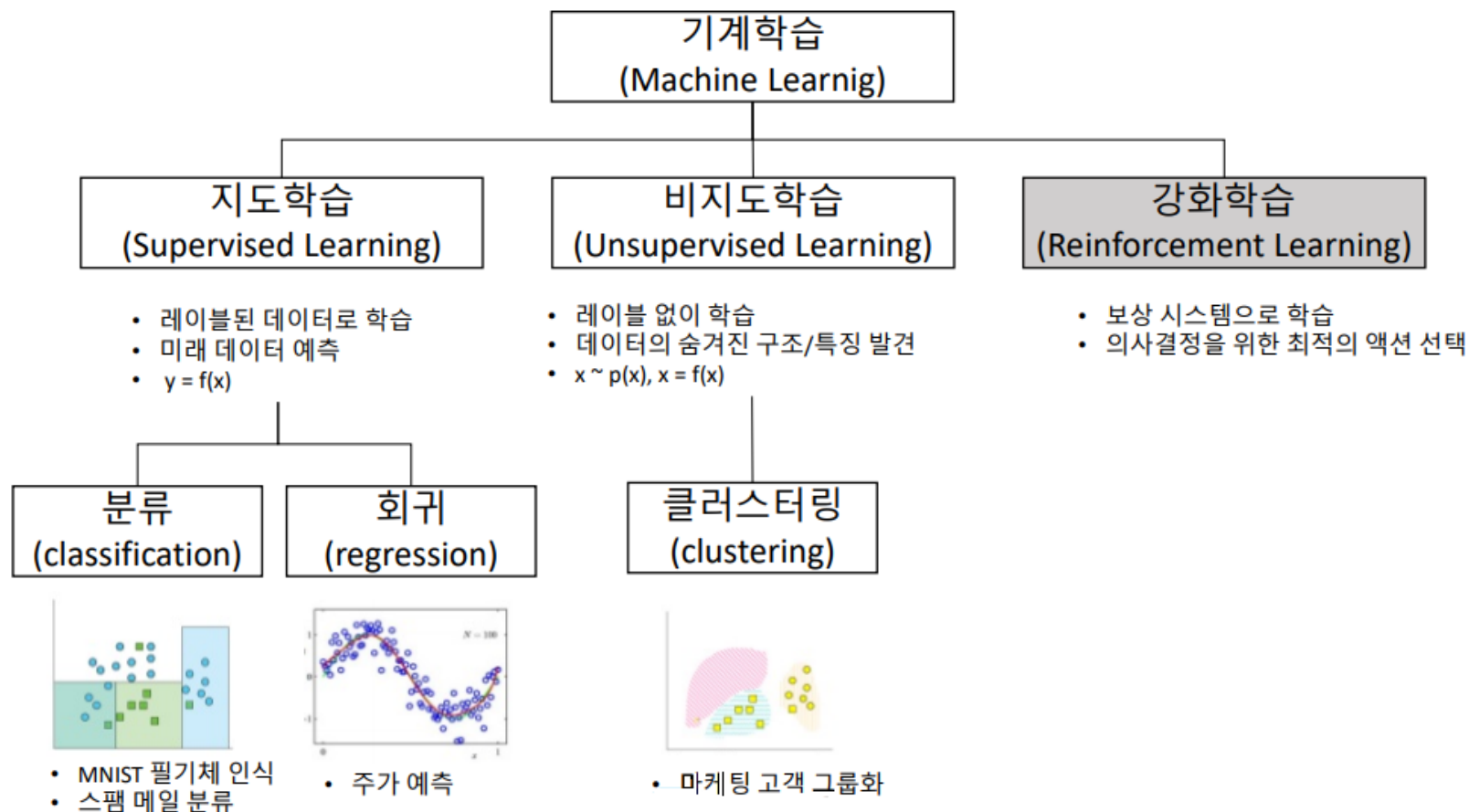


9강. AI 모델 분석 및 설계2 (이진 분류, Logistic Regression)

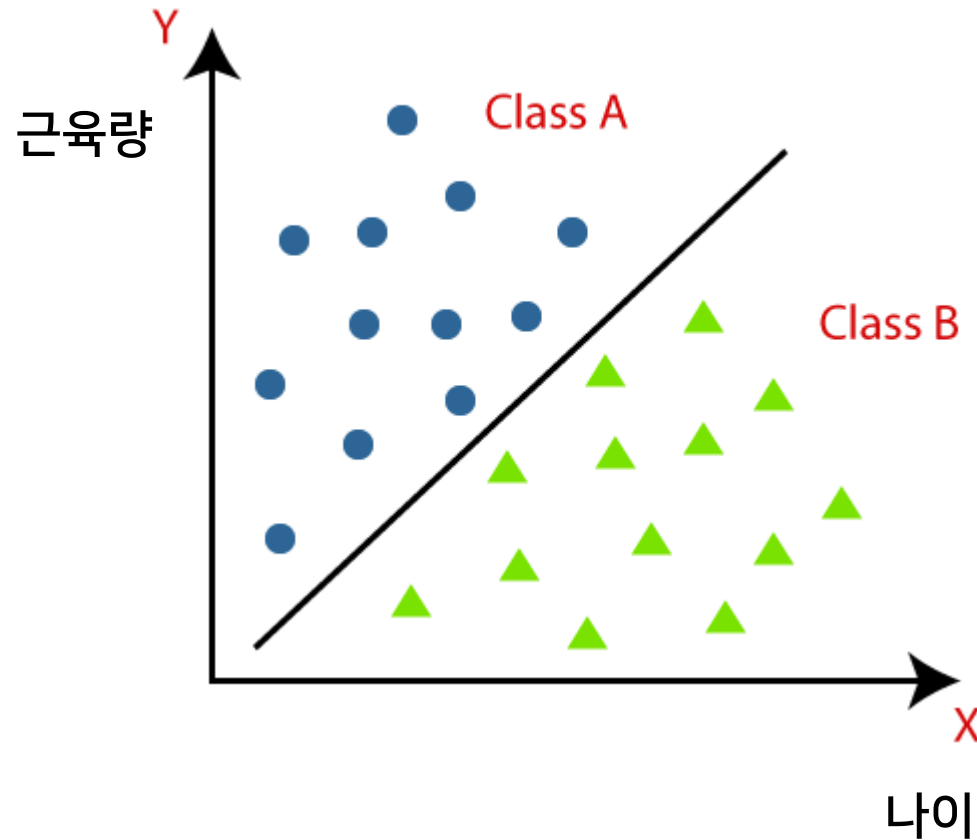
머신 러닝 (Machine Learning)

- Supervised learning / Unsupervised learning / Reinforcement learning



지도학습 (Supervised Learning)

- 분류 (Classification)



회귀 (Regression)

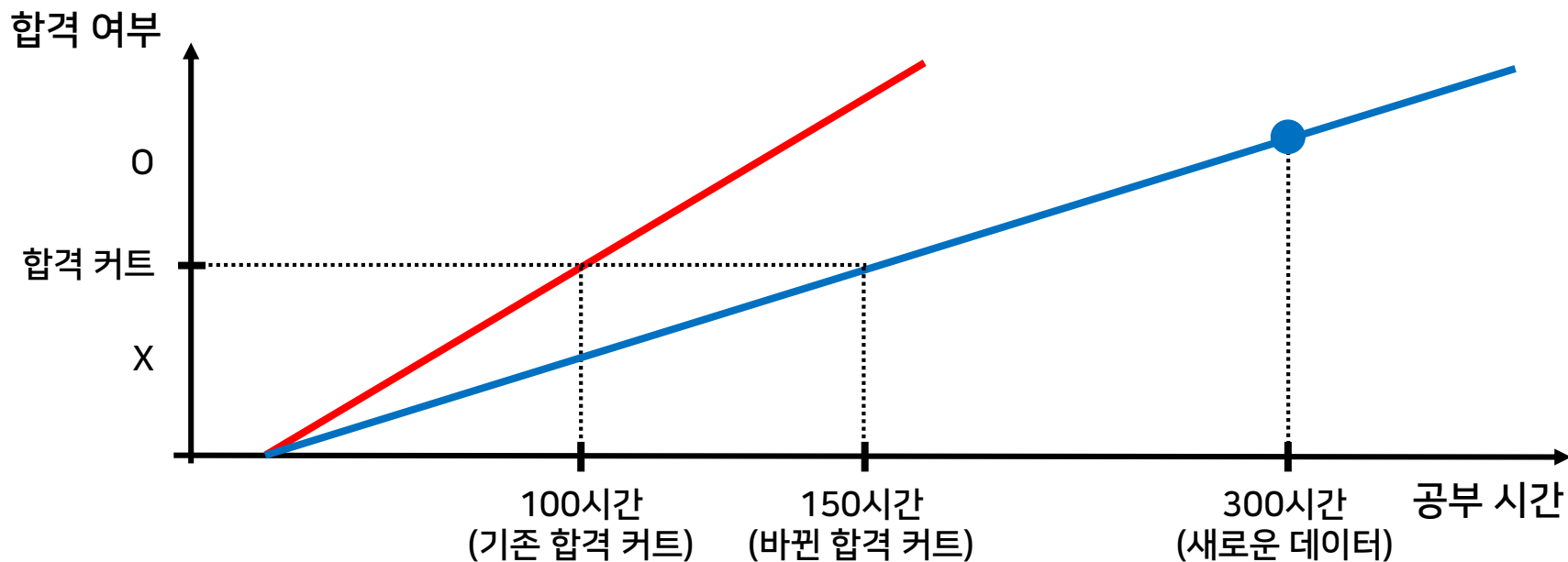
- 입력값: 연속값(실수형), 이산값(범주형) 등 모두 가능
- 출력값: 연속값(실수형)
- 모델 형태: 일반적인 함수 형태 (e.g. $h_{\theta}(x) = \theta_0 + \theta_1 x$)

분류 (Classification)

- 입력값: 연속값(실수형), 이산값(범주형) 등 모두 가능
- 출력값: 이산값(범주형)
- 모델 형태:
 - 이진 분류: 시그모이드(Sigmoid) 함수
 - 다중 분류: 소프트맥스(Softmax) 함수를 꼭 포함

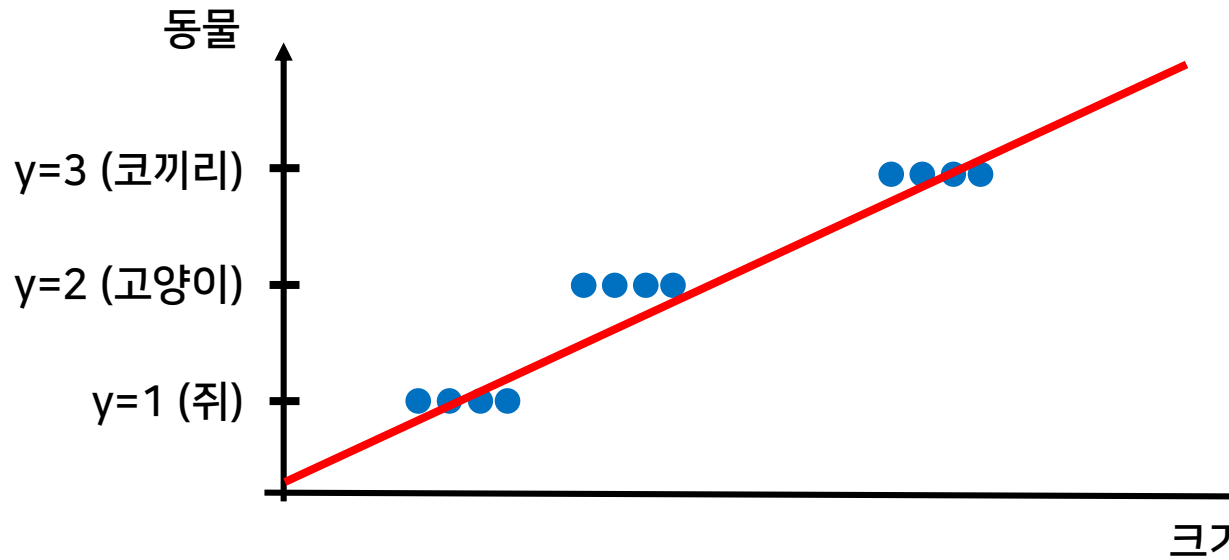
☑ 선형 회귀는 예외적인 데이터에 민감하게 반응함

- 선형회귀로도 분류는 할 수 있음
 - ex) 공부 시간이 100시간이 넘으면 합격, 100시간이 넘지 않으면 불합격으로 분류하면 됨
- 문제점: 1개의 예외 데이터에 너무 민감하게 반응함
 - ex) 공부 시간이 300시간인 학생 데이터가 나타나면 선형 모델이 확 바뀜



☑ 선형 회귀는 범주의 순서 매기기에 따른 오해가 발생함

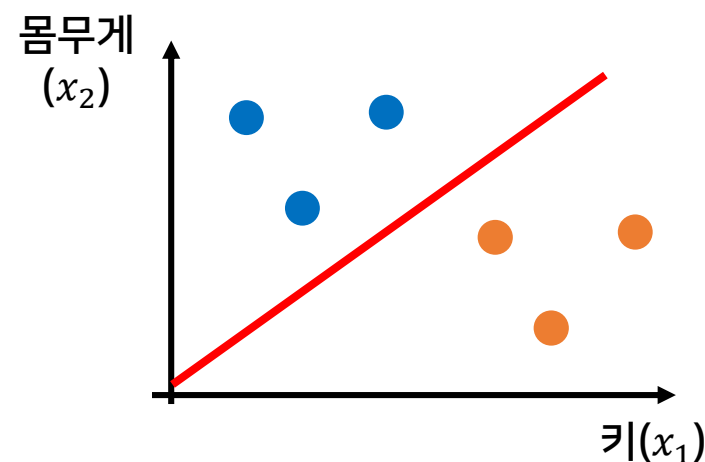
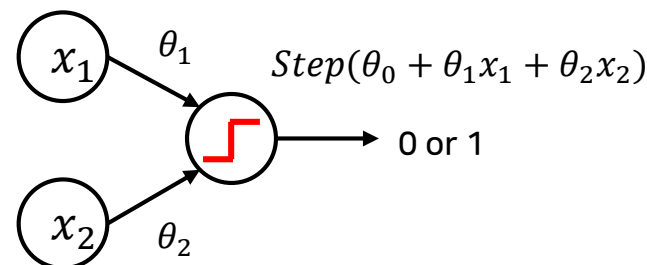
- 일반적인 선형 회귀에서는 라벨의 순서(크기)에 따라 결과(오차)가 달라짐
 - 쥐를 고양이로 잘못 예측하면? $MSE = (2 - 1)^2 = 1$
 - 쥐를 코끼리로 잘못 예측하면? $MSE = (3 - 1)^2 = 4$



- 따라서 (MSE가 아닌) 다른 손실 함수나 (선형이 아닌) 다른 모델이 필요함

✍ 학습 과정

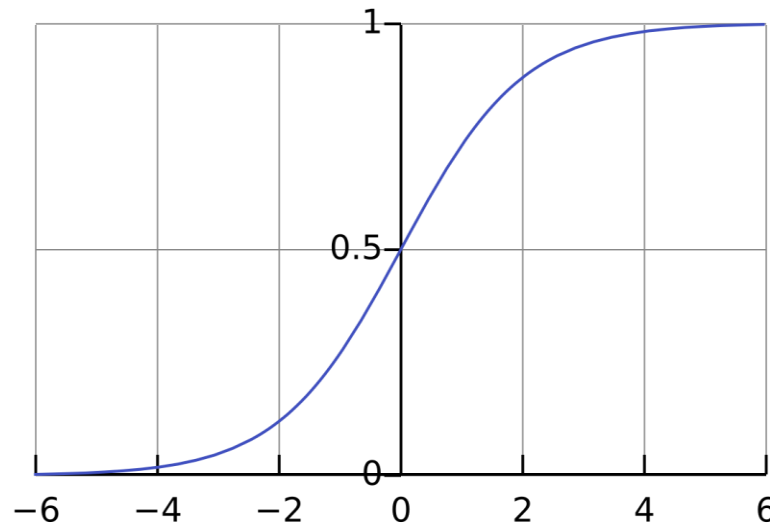
- 데이터(입력 및 출력) 모으기
 - 입력(다중 입력) : 키, 몸무게
 - 출력(이진 분류) : 0 또는 1
- 모델(비선형 함수) 만들기
 - unit step function (출력이 0 or 1)
- 모델 학습시키기
 - 분류 경계가 선형이 됨(=선형 분류)
- 모델 테스트하기
 - Unit step function 문제점이 존재
 - 엄청 뽀뽀하게 분류함
 - 미분이 불가능함 (Gradient Descent 사용 불가)
- 그래서 Sigmoid function을 사용!!!!
 - 부드럽게 분류 (0~1 사이의 확률로 해석 가능)
 - 미분 가능 (Gradient Descent 사용 가능)



3. Logistic Function (=sigmoid function)

✍ 시그모이드 함수

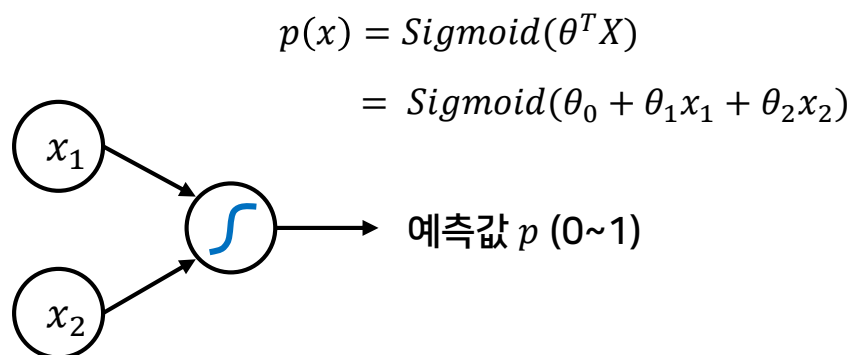
- $\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$
- 무한대 범위 입력을 0~1 사이로 변경해줌
 - 함수의 출력 값이 항상 0 이상 1 이하임
- $x = 0$ 일 때 함수의 출력값은 0.5
- [실습] 시그모이드 함수를 MATLAB에서 그려보자.



$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

로지스틱 회귀 모델을 학습한다는 것?

- 데이터를 가장 잘 분류하는 '경계 선'을 찾을 건데, 예측 값과 실제 값의 차이를 나타내는 Loss를 구해서 Loss를 줄여나가면서 '선에 대한 파라미터'를 학습하면 됨
- 결국 로지스틱 회귀도 선형 회귀와 마찬가지로 **Loss만 구하면** Gradient Descent 사용하면 됨
 - MSE(Mean Squared Error)를 사용하면 될까?



Logistic Regression

수렴할 때까지 반복{
 $\theta_0 = \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$
 $\theta_1 = \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$
 $\theta^T X$ 업데이트
}

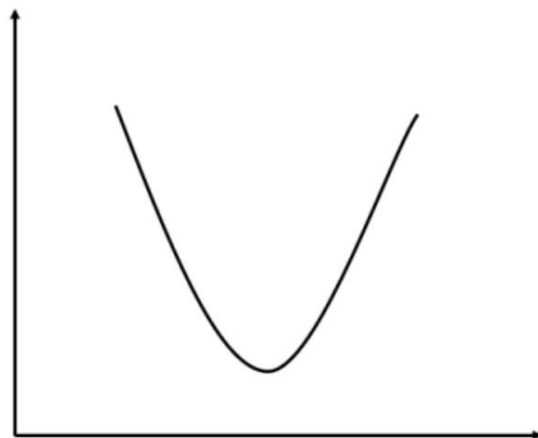
Gradient Descent

☑ Mean Squared Error를 사용할 경우

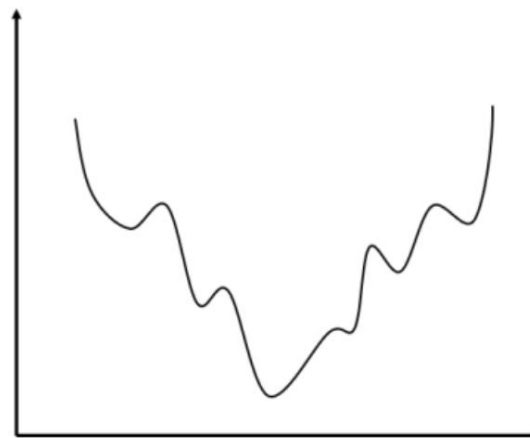
- Loss function으로 MSE를 사용하면, 선형 회귀와 다르게 non-convex function이 됨
 - 예측 값이 선형이 아니기(sigmoid) 때문에
- non-convex function은 gradient descent를 통해 global optimum을 찾아갈 수 없음

$$p(x) = \text{sigmoid}(\theta^T X) = \frac{1}{1 + e^{-(\theta^T X)}}$$

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (p(x^i) - y^i)^2$$



기존 선형 회귀 비용함수



로지스틱 함수 비용함수

✍ Binary Cross Entropy 를 사용하자!

- 이진 분류에 사용되는 손실 함수
- $y = 1$ 인 값을 예상할 때
 - 예측한 값인 p 가 1에 가깝게 나오면, $loss \cong 0$
 - 예측한 값인 p 가 0에 가깝게 나오면, $loss \cong \infty$

이 되도록 손실 함수를 정의하면 됨

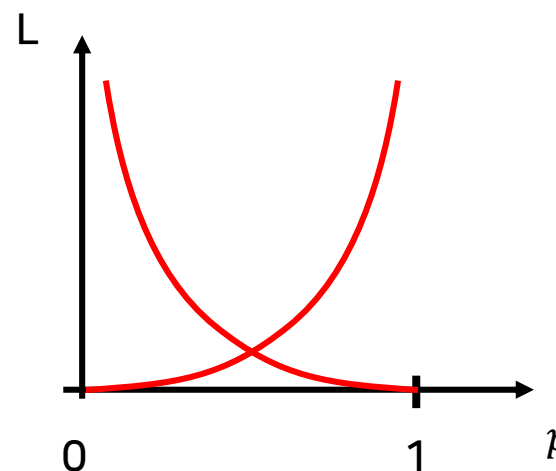
- cost function

$$J(\theta) = \begin{cases} -\log(p), & \text{if } y = 1 \\ -\log(1-p), & \text{if } y = 0 \end{cases}$$
$$= -\log(p^y (1-p)^{1-y})$$

- 모든 데이터에 대해서 합하면?

$$J(\theta) = \sum_{i=1}^m \log(p_i^y (1-p_i)^{1-y})$$

Binary Cross Entropy



Binary Cross Entropy (Loss function) 사용된 과정 설명

- BCE가 사용된 과정 설명
- 신경망 출력을 p , 정답을 y 라고 하고 데이터를 순서대로 넣어보자.
 - $y_1 = 1$ 이면? $P(y_1)$ 은 p 가 1일 확률 $\rightarrow p$ 를 크게 하자
 - $y_2 = 0$ 이면? $P(y_2)$ 은 p 가 0일 확률 $\rightarrow p$ 를 작게 하자 = $1 - p$ 를 크게 하자
 - ...
 - 따라서, $p^y(1-p)^{1-y}$ 를 크게 하자로 통일이 가능
- 각 데이터는 독립시행이므로 확률의 곱을 크게하면 됨
 - $p_1^y(1-p)_1^{1-y} p_2^y(1-p)_2^{1-y} \dots p_m^y(1-p)_m^{1-y}$
 - 문제점: 0~1 사이 값을 계속 곱하는 것이 되어 점점 0에 수렴함
- 해결방법: log를 취하면 곱셈이 덧셈으로 바뀔 수 있음
 - $L = -\log p_1^y(1-p)_1^{1-y} p_2^y(1-p)_2^{1-y} \dots p_m^y(1-p)_m^{1-y} = \sum_{i=1}^m \log(p_i^y(1-p)_i^{1-y})$

Binary Cross Entropy

✍ Logistic Regression 이름의 참 의미 (왜 Regression인가?)

- Logit을 Linear Regression으로 구한 것이고, 단지 loss 값을 얻기 위해 sigmoid를 통과시켜서 $\text{logit}(-\infty \sim \infty)$ 을 확률($0 \sim 1$)로 바꾼 것
- 로지스틱 함수(Sigmoid)가 정의되는 과정
 - odds ratio (승산비) : 베르누이 시도에서 1이 나올 확률 p 와 0이 나올 확률 $1-p$ 의 비율

- $odd = \frac{p}{1-p}$

- logit ($-\infty \sim \infty$) : 승산비를 로그 변환한 것

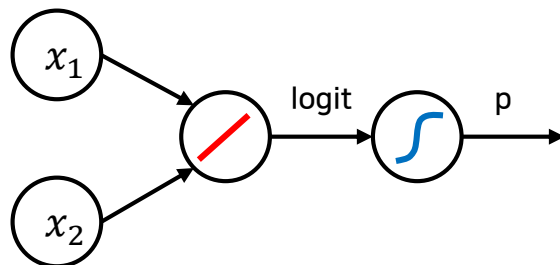
※ 왜 이런 짓을 해?
무한대 범위를 확률로 바꾸기 위해서!
로그와 확률의 관계식을 세우는 것

- $logit = \log\left(\frac{p}{1-p}\right)$

- logistic function ($0 \sim 1$) : logit의 역함수

- $e^{logit} = \frac{p}{1-p} \gg e^{-logit} = \frac{1-p}{p} = \frac{1}{p} - 1 \gg \boxed{p = \frac{1}{1+e^{-logit}}}$

Sigmoid



※ 'Sigmoid는 단순히 BCE Loss를 얻기 위한 것이다'로 해석이 가능

✍ BCE를 어떻게 최소화할까?

■ 경사 하강법 (Gradient Descent)

- 다시 원래 문제로 돌아가서...

- Cost function

$$J(\theta) = \sum_{i=1}^m \log(p_i^y (1 - p_i)^{1-y})$$

- Goal

$$\underset{\theta}{\text{minimize}} J(\theta)$$

- Algorithm (Gradient Descent)

- 파라미터가 두 개라면, 두 파라미터에 대해 편미분을 동시에 수행해야 함

수렴할 때까지 반복{

$$\theta_0 = \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta)$$

$$\theta_1 = \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta)$$

$\theta^T X$ 업데이트

}

편미분 &
Chain rule

$$\theta^T X = \theta_0 + \theta_1 x$$

수렴할 때까지 반복{

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (p - y^i) x_0^i$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (p - y^i) x_1^i$$

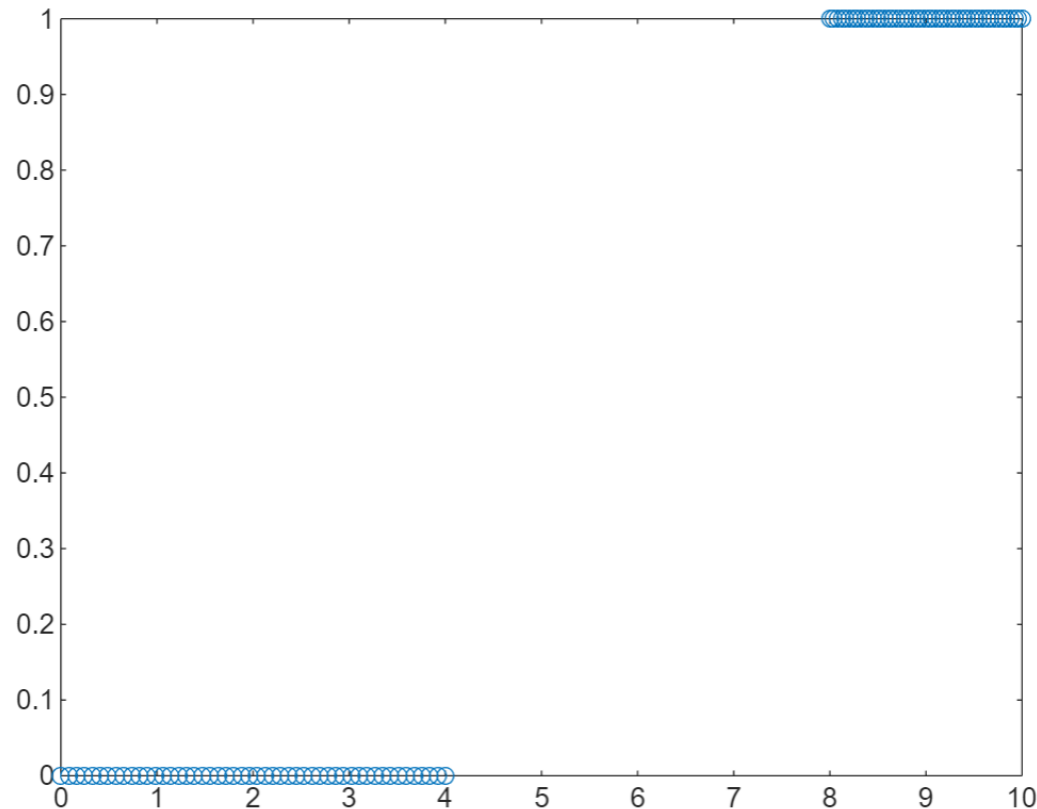
$\theta^T X$ 업데이트

}

✍ 분류에 적합한 데이터셋 준비

- 첫 번째 클래스
 - $x_1 = 0 \sim 4$ 사이의 데이터 50개
 - $y_1 = 0$
- 두 번째 클래스
 - $x_2 = 8 \sim 10$ 사이의 데이터 50개
 - $y_2 = 1$

```
1  clc;  
2  clear;  
3  close all;  
4  
5  x1 = linspace(0, 4, 50)';  
6  y1 = zeros(50, 1);  
7  x2 = linspace(8, 10, 50)';  
8  y2 = ones(50, 1);  
9  data = [x1, y1; x2, y2];  
10 x = data(:, 1);  
11 y = data(:, 2);  
12 plot(x, y, 'o');
```



선형 회귀(지난 시간)를 적용하여 분류해보자

- Hypothesis

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

- Loss Function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$$

- Algorithm

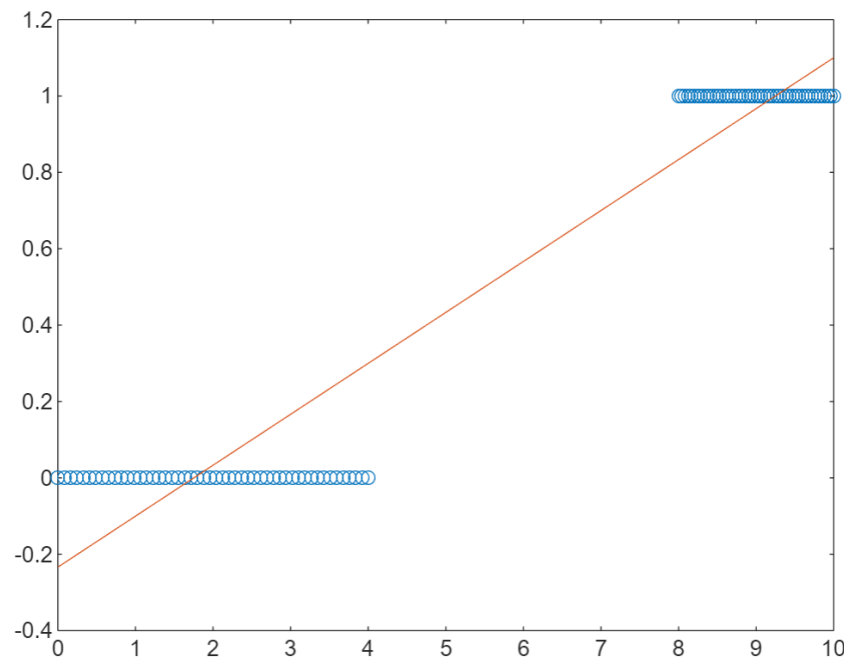
수렴할 때까지 반복{

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i) x_0^i$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i) x_1^i$$

$h_{\theta}(x)$ 업데이트

}



비선형 회귀(로지스틱 회귀)를 적용하여 분류해보자

▪ Hypothesis

$$p_{\theta}(x) = \frac{1}{1 + e^{-(\theta^T X)}}$$

▪ Loss Function

$$J(\theta) = \sum_{i=1}^m \log(p_i^y (1 - p_i)^{1-y})$$

▪ Algorithm

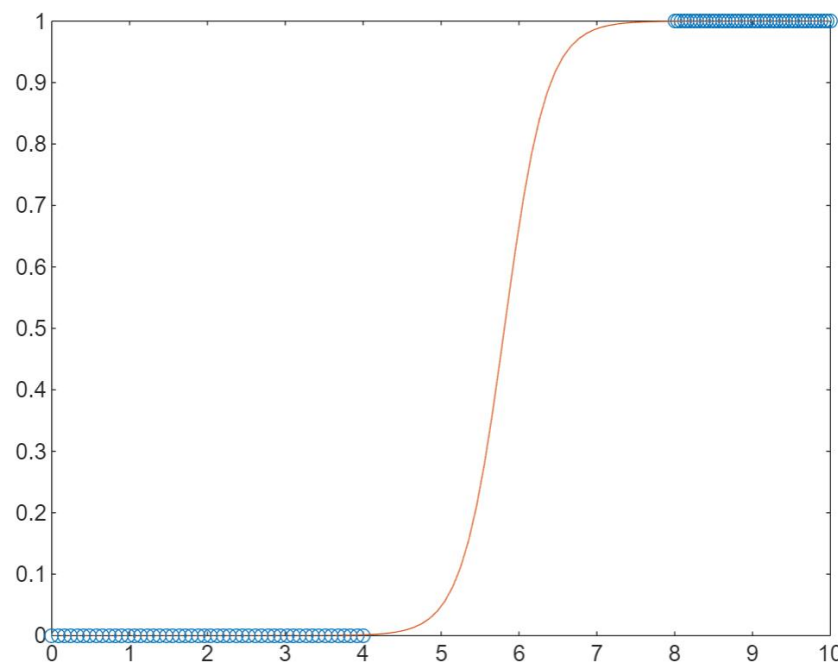
수렴할 때까지 반복{

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (p(x^i) - y^i) x_0^i$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (p(x^i) - y^i) x_1^i$$

$\theta^T X$ 업데이트

}



비선형 회귀(로지스틱 회귀)를 적용하여 분류해보자

▪ Hypothesis

$$p_{\theta}(x) = \frac{1}{1 + e^{-(\theta^T X)}}$$

▪ Loss Function

$$J(\theta) = \sum_{i=1}^m \log(p_i^y (1 - p_i)^{1-y})$$

▪ Algorithm

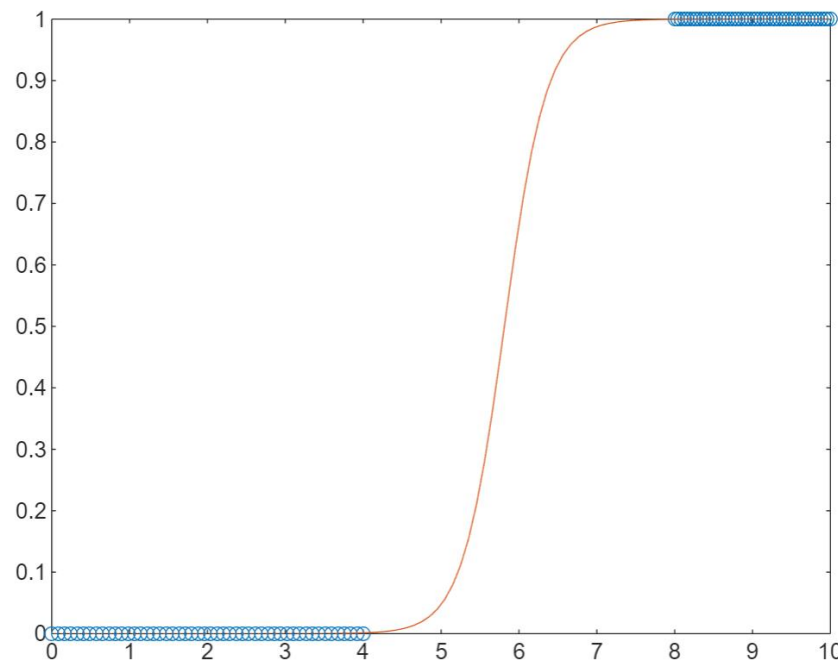
수렴할 때까지 반복{

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (p(x^i) - y^i) x_0^i$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (p(x^i) - y^i) x_1^i$$

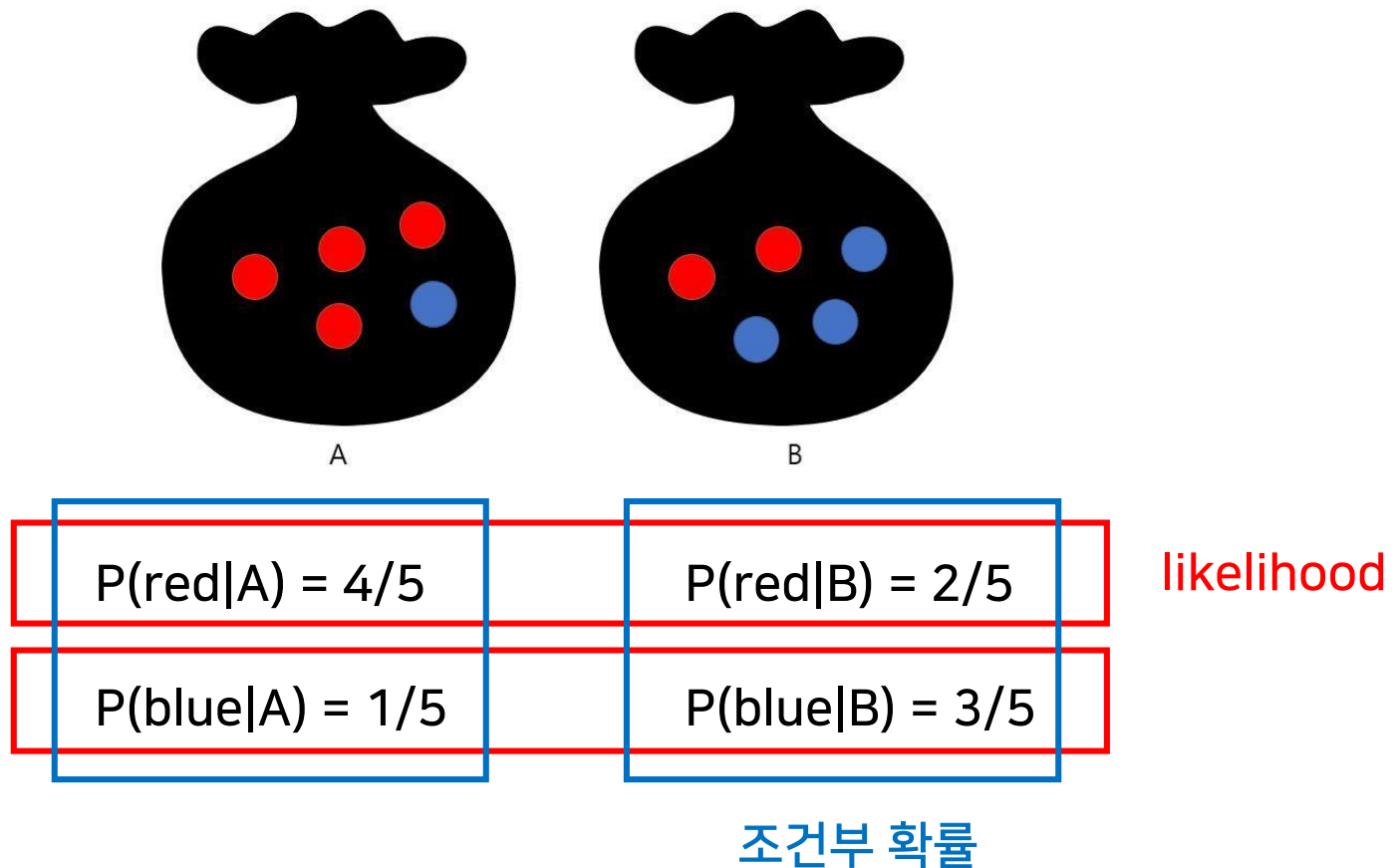
$\theta^T X$ 업데이트

}



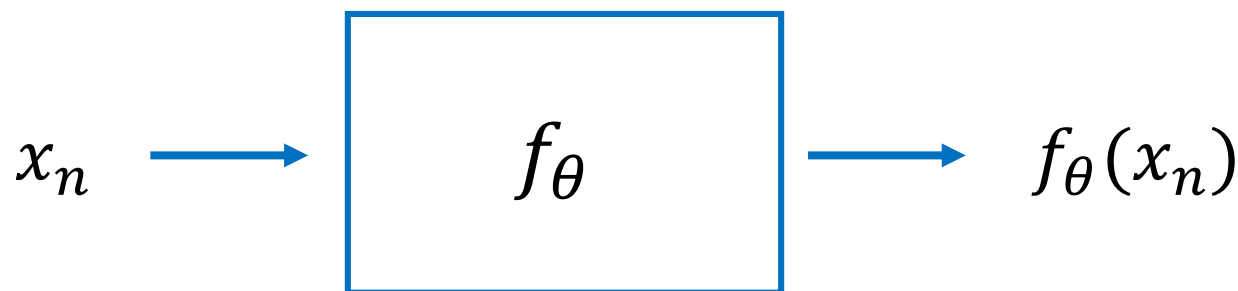
Maximum Likelihood Estimation (MLE)

- Likelihood (가능도, 우도): Measurement를 보고 가능성을 추정하에 알아냄
- Maximum Likelihood (최대 우도): Measurement를 보고 가능성이 최대인 것을 뽑기
 - 빨간 공이 나왔다면 A와 B 중 어떤 것일 확률이 높은가? 당연히 A!



✍ Maximum Likelihood Estimation (MLE)

- 예측한 값($f_{\theta}(x_n)$)이 나왔을 때 정답(y_n)일 확률($P(y_n|f_{\theta}(x_n))$)을 최대로 만드는 파라미터(θ)를 찾는 것
- 여러 개 데이터가 존재하면 독립시행이므로 곱하면 0으로 수렴함
 - 따라서 Negative Log Likelihood (NLL) 수행: Log를 취하고 -1을 곱함



$$\operatorname{argmax}_{\theta} P(y_n | f_{\theta}(x_n))$$

Q & A

Contacts: yhkim@dongseo.ac.kr
(UIT211)